

HelixCode extensions (plugins) — the honest guide

HelixCode extensions (plugins) — the honest guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Scope:** How code-server's extension (plugin) system works in HelixCode — which marketplace it really uses, how to install/list/remove extensions (UI + CLI + local .vsix), how the gallery is configured, and how to troubleshoot. **Authority:** constitution §11.4.6 (no-guessing — every fact below is verified against the running binary and the latest official docs; see *Sources verified*), §11.4.10 (no secrets), §11.4.65 (.md+.html+.pdf in sync).

Verified against this install: code-server 4.117.0 (with Code 1.117.0), the exact binary HelixCode runs. Command/flag names below are copied from code-server --help on this host, not from memory.

TL;DR — the marketplace reality

- HelixCode is **code-server**, a third-party (non-Microsoft) build of VS Code.
- Its **default extension marketplace is Open VSX** — an **open-source, vendor-neutral** registry operated by the **Eclipse Foundation** (EPL-2.0). It is **NOT** the Microsoft “VS Code Marketplace”.
- code-server **cannot** use Microsoft's Marketplace: Microsoft's Marketplace **Terms of Use restrict it to Microsoft's own Visual Studio products** — using it from a third-party editor would violate those terms. This guide does **not** provide a way around that restriction; it documents the supported Open VSX path.
- Practical effect: **most** popular extensions are on Open VSX. A few Microsoft **closed-source** extensions are **not** (notably **Live Share** and the **Remote** family — Remote-SSH / Dev

Containers / WSL). Third-party alternatives usually exist on Open VSX.

1. Why Open VSX (and not the Microsoft Marketplace)

What Open VSX is. Per the Eclipse project, Open VSX is a “*vendor-neutral open-source alternative to the Visual Studio Marketplace*” — a server + web app + publishing CLI, licensed **EPL-2.0**, operated by the **Eclipse Foundation**, with the public instance at <https://open-vsx.org>. It serves the same *kind* of VS Code extensions, through an independent, non-proprietary channel.

Why code-server defaults to it. The Microsoft VS Code Marketplace’s Terms of Use state (quoted in the code-server FAQ) that Marketplace Offerings are “*intended for use only with Visual Studio Products and Services*”. code-server is not a Microsoft product, so it **legally cannot** point at Microsoft’s Marketplace, and instead ships **Open VSX** as the default gallery. This is a licensing boundary, stated here as fact — **HelixCode does not bypass it**.

What you may miss. Two Microsoft **closed-source** extension sets are not on Open VSX and are unavailable in code-server:

Not available	Practical note
Live Share	Microsoft closed-source; no Open VSX equivalent from MS.
Remote — Remote-SSH, Dev Containers, WSL	Microsoft closed-source. code-server already runs <i>on</i> the host as your real account (see AUTH.md), so remote-into-host is largely moot; the integrated terminal + Open Folder already have full host access.

Everything else — language servers, linters, themes, formatters, YAML/JSON tools, etc. — is generally on Open VSX and installs normally.

2. Installing, listing, and removing extensions

2.1 From the editor UI (easiest)

1. Open the **Extensions** view — the squares icon in the Activity Bar, or Ctrl+Shift+X.
2. **Search** by name/id and click **Install**. Results come from **Open VSX**.
3. To remove: find the installed extension → gear icon → **Uninstall**.

The UI writes into HelixCode's **private** extensions directory (below), so it never disturbs any other VS Code / code-server you run on the host.

2.2 HelixCode's extensions directory (important)

The HelixCode systemd unit (deploy/systemd/helix-code-server.service) runs code-server with **private** state paths so this instance never shares the operator's global ~/.vscode / ~/.local/share/code-server:

```
--user-data-dir    ~/.local/share/helixcode/code-server
--extensions-dir   ~/.local/share/helixcode/code-server/extensions
```

That directory is the **source of truth** for what the running editor loads. On this host it currently holds, for example:

```
~/.local/share/helixcode/code-server/extensions/
  adamraichu.pdf-viewer-1.1.2-universal/      # an installed
extension
  extensions.json                             # code-server's
install registry
```

2.3 From the command line

code-server exposes the standard VS Code extension CLI (verified on this host — code-server --help):

Flag	Meaning
--install-extension <publisher.name>[@version]	Install/update from the gallery (Open VSX). e.g. redhat.vscode-yaml, or pin ms-vscode.foo@1.2.3.
--install-extension <path-to.vsix>	Install a local .vsix (see §4).
	Remove an installed extension.

Flag	Meaning
<code>--uninstall-extension</code> <code><publisher.name></code>	
<code>--list-extensions</code>	List installed extension ids.
<code>--show-versions</code>	(with <code>--list-extensions</code>) also print @version.
<code>--force</code>	Skip prompts when installing/ updating.
<code>--enable-proposed-api</code> [<code><id></code> ...]	Opt an extension into proposed APIs.

There is NO `--extensions-gallery` CLI flag in code-server 4.117.0 (confirmed by `code-server --help`). The gallery is configured by the `EXTENSIONS_GALLERY` **environment variable** or `product.json` — see §3. Do not expect a `--extensions-gallery` switch; it does not exist in this version.

CRITICAL for HelixCode — target the same dirs. A *bare* `code-server --install-extension foo` uses code-server’s **default** paths (`~/.local/share/code-server/extensions`), which the HelixCode **service does not load**. To install into the directory the running HelixCode editor actually uses, pass the **same** `--extensions-dir` (and `--user-data-dir`) the service uses:

```
CS=~/.local/bin/code-server
    # the code-server binary on this host

"$CS" \
  --extensions-dir ~/.local/share/helixcode/code-server/
    extensions \
  --user-data-dir  ~/.local/share/helixcode/code-server \
  --install-extension redhat.vscode-yaml

# verify it landed:
"$CS" \
  --extensions-dir ~/.local/share/helixcode/code-server/
    extensions \
  --user-data-dir  ~/.local/share/helixcode/code-server \
  --list-extensions --show-versions
```

Then **reload the editor** (browser: *Developer: Reload Window*, or restart the service so the extension host re-scans):

```
systemctl --user restart helix-code-server
```

Installing from the **UI** (§2.1) already writes to the correct HelixCode dir and hot-loads without a restart — the `--extensions-dir` caveat only applies to **CLI** installs run in a separate process.

3. How the gallery is configured (and how to point at another one)

3.1 Where the default comes from

code-server ships **Open VSX** as the built-in gallery. Verified on this install:

- The bundled VS Code `product.json` (`~/.local/lib/code-server-4.117.0-linux-amd64/lib/vscode/product.json`) has `"extensionsGallery": null` — it does **not** hard-code a gallery.
- code-server injects the **Open VSX** endpoints as its default (these strings are compiled into the code-server distribution and match the Eclipse “Using Open VSX in VS Code” doc):

```
{
  "serviceUrl": "https://open-vsx.org/vscode/gallery",
  "itemUrl":    "https://open-vsx.org/vscode/item"
}
```

- The HelixCode systemd unit does **not** set `EXTENSIONS_GALLERY`, so the running editor uses this Open VSX default.

You can print the currently-configured gallery at any time with the read-only helper shipped in this repo:

```
scripts/show-extension-gallery.sh
```

3.2 Precedence (what wins)

code-server resolves the gallery in this order:

1. `EXTENSIONS_GALLERY` environment variable (if set) — highest priority; a JSON blob matching VS Code’s `extensionsGallery` shape.
2. `product.json extensionsGallery` (if non-null).
3. Otherwise code-server’s **built-in Open VSX default** (§3.1).

3.3 Pointing at an ALTERNATE or SELF-HOSTED Open-VSX-compatible gallery

If you run (or trust) another registry that implements the **VS Code Extension Gallery API** — for example a **self-hosted Open VSX** instance, or an internal mirror — you can point code-server at it with `EXTENSIONS_GALLERY`. This is an **operator decision**, so it is not wired by default.

```
export EXTENSIONS_GALLERY='{
  "serviceUrl": "https://my-openvsx.example.com/vscode/gallery",
```

```
"itemUrl": "https://my-openvsx.example.com/vscode/item"
}'
# then start / restart code-server in an environment that has this
  exported.
```

To make it persistent for the HelixCode service, add it to the service environment (operator-applied), e.g. a systemd --user drop-in:

```
mkdir -p ~/.config/systemd/user/helix-code-server.service.d
cat > ~/.config/systemd/user/helix-code-server.service.d/20-
  gallery.conf <<'EOF'
[Service]
Environment=EXTENSIONS_GALLERY={"serviceUrl":"https://my-
  openvsx.example.com/vscode/gallery","itemUrl":"https://my-
  openvsx.example.com/vscode/item"}
EOF
systemctl --user daemon-reload
systemctl --user restart helix-code-server
```

Licensing caveat (read this). The EXTENSIONS_GALLERY mechanism is a generic pointer. Do **NOT** set it to Microsoft's VS Code Marketplace endpoints: Microsoft's Marketplace Terms of Use restrict that Marketplace to Microsoft's own products, and pointing a third-party editor at it violates those terms. Use it only for **Open VSX** (the public instance) or an **Open-VSX-compatible** registry you are entitled to use. HelixCode ships no tooling to switch to the Microsoft Marketplace, by design.

4. Installing a local .vsix

You can side-load an extension packaged as a .vsix file (e.g. one you built, or downloaded from a source you trust). Point --install-extension at the file path instead of an id — again with HelixCode's dirs:

```
CS=~/.local/bin/code-server
"$CS" \
  --extensions-dir ~/.local/share/helixcode/code-server/
  extensions \
  --user-data-dir ~/.local/share/helixcode/code-server \
  --install-extension ./some-extension-1.0.0.vsix
systemctl --user restart helix-code-server
```

Equivalent in the **UI**: Extensions view → the ... menu → **Install from VSIX...** → pick the file. (The UI path writes to the correct dir automatically.)

5. Troubleshooting

Evidence base. The end-to-end install + loadability behavior described here is exercised by the anti-bluff suite tests/types/extensions_auth.sh (§11.4.169 e2e/full-automation layer). It installs a real extension from **Open VSX** into a throwaway --extensions-dir, asserts the CLI reports it (--list-extensions --show-versions), checks the on-disk package.json + entry file, boots a throwaway code-server and proves the **extension host** starts and watches the registry, and never touches your live extensions dir. Its captured evidence lands under qa-results/tests/extensions_auth/<run-id>/ — the concrete proof for the claims below.

Symptom	Likely cause	Fix
CLI-installed extension not visible in the running editor	Installed into code-server's default dir, not HelixCode's private dir.	Re-run the install with --extensions-dir ~/.local/share/helixcode/code-server/extensions --user-data-dir ~/.local/share/helixcode/code-server (§2.3), then reload/restart.
Extension installs but doesn't load	Editor didn't re-scan; or the extension needs a proposed API / newer Code version.	<i>Developer: Reload Window</i> in the browser, or systemctl --user restart helix-code-server. Check the extension's required VS Code version against 1.117.0.
"Extension not found" / not in search	It's a Microsoft closed-source extension (Live Share / Remote), or simply not published to Open VSX.	Look for an Open VSX alternative; MS closed-source extensions are unavailable (§1).
Search returns nothing / installs time out	Gallery unreachable — open-vsx.org (or your custom gallery) not reachable from the host.	Check egress: curl -sS -o /dev/null -w '%{http_code}\n' https://open-vsx.org/api/redhat/vscode-yaml should print 200. If it's 000, the host can't reach the gallery (proxy/firewall/DNS). The suite reports this as an honest SKIP (network_unreachable_external), never a false pass.
	—	

Symptom	Likely cause	Fix
Want to confirm which gallery is active		Run <code>scripts/show-extension-gallery.sh</code> (read-only).
Custom <code>EXTENSIONS_GALLERY</code> ignored	Not exported into the service environment.	Add it via a <code>systemd --user</code> drop-in (§3.3), <code>daemon-reload</code> , <code>restart</code> .

Sources verified

Verified **2026-07-01** against the latest official sources (§11.4.99) and the running binary on this host (§11.4.6 / §11.4.123):

- code-server FAQ — marketplace, MS Marketplace Terms of Use restriction, `--install-extension`, `.vsix`, `EXTENSIONS_GALLERY`, Live Share / Remote unavailability: <https://coder.com/docs/code-server/FAQ>
- Open VSX / Eclipse project README — “vendor-neutral open-source alternative to the Visual Studio Marketplace”, EPL-2.0, operated by the Eclipse Foundation: <https://github.com/eclipse/openvsx> · <https://raw.githubusercontent.com/eclipse/openvsx/master/README.md>
- Eclipse “Using Open VSX in VS Code” — `serviceUrl` <https://open-vsx.org/vscode/gallery>, `itemUrl` <https://open-vsx.org/vscode/item>: <https://github.com/eclipse/openvsx/wiki/Using-Open-VSX-in-VS-Code>
- coder/code-server discussion — changing the default marketplace via `EXTENSIONS_GALLERY`: <https://github.com/coder/code-server/discussions/5017>
- Open VSX Registry (public instance): <https://open-vsx.org>
- Local rock-solid evidence (this host): `code-server --version` → 4.117.0 with Code 1.117.0; `code-server --help` (extension flags; **no** `--extensions-gallery` flag); `deploy/systemd/helix-code-server.service` (`--extensions-dir/--user-data-dir` paths, no `EXTENSIONS_GALLERY`); `.../lib/vscode/product.json` (`extensionsGallery: null`); live extensions dir holding `adamraichu.pdf-viewer-1.1.2-universal`; `suite tests/types/extensions_auth.sh`.

Negative finding (§11.4.99(B)): `code-server 4.117.0` exposes **no** `--extensions-gallery` CLI flag; the only supported gallery-override mechanisms are the `EXTENSIONS_GALLERY` environment variable and `product.json`.